

Phantom-Conditioned Nested Sampling

Joshua G. Albert^{1,2*}

¹Leiden Observatory, Leiden University, PO Box 9513, Leiden, 2300,
The Netherlands.

²Department of Astronomy, California Institute of Technology, CA
91125, Pasadena, USA.

Corresponding author(s). E-mail(s): albert@strw.leidenuniv.nl;

Abstract

Nested sampling estimates the evidence by assigning prior volumes to an ordered sequence of likelihood contours. Markov chain constrained-prior samplers, which are used in high-dimensional problems, generate many intermediate states before producing the next classic sample. These intermediate states are discarded because their correlation prevents them from being inserted into the ordered NS sequence without changing its order-statistic law. This paper introduces a novel method of using them to improve evidence estimation, by formulating NS in a Bayesian way and conditioning on phantom samples as Monte Carlo observations. We then present the open-source software package, JAXNS v3, and its implementation choices. We validate JAXNS on difficult evidence and posterior-estimation benchmarks for accuracy and performance.

Keywords: Nested Sampling, Bayesian evidence, order statistics, exponential races, Monte Carlo, Markov-chain, phantom samples

1 Introduction

Nested sampling (NS; [Skilling, 2004](#)) rewrites Bayesian evidence estimation as a one-dimensional compression problem. For standard background, terminology, and a broad review of modern implementations we refer to [Buchner \(2023\)](#); here we only introduce the notation needed for the present construction. Let $(\mathcal{A}, \Sigma, p)$ be a probability space and let $L : \mathcal{A} \rightarrow [0, \infty)$ be a measurable likelihood or likelihood-like score. The evidence

is

$$Z = \int_{\mathcal{A}} L(x) \, dp(x). \quad (1)$$

Nested sampling proceeds by constructing likelihood contours and assigning prior volumes to them. The computational difficulty is not the one-dimensional quadrature once these volumes are known, but the stochastic inference of the volume sequence.

The present paper introduces a version of nested sampling in which the volume sequence is treated as a random variable with a race-induced prior that can be conditioned on auxiliary Monte Carlo information. The auxiliary information comes from *phantom samples*: intermediate states produced by a constrained-prior sampler while generating a classic sample. Phantom samples are correlated and thus cannot be used in the classic shrinkage formalism. The reason can be understood in two ways. Under the race view, it is because the races are no longer independent, so winner rate is no longer the sum of all rates. Under the order-statistic view, it is because the correlation gives non-uniform probability of some permutations of orderings of likelihoods. In this paper, we will adopt the race view, which is described in Section 2.

Despite their inadequacy for classic shrinkage, phantom samples are still drawn from the same target distribution, and thus can be used for unbiased Monte Carlo expectation estimators. We will use them to form unbiased estimates of constrained-prior probabilities,

$$\mathbb{P}_{x \sim p(\cdot | L > a)}(L(x) > b) = \frac{X_b}{X_a}, \quad (2)$$

which are precisely the shrinkage ratios supplied by the classic order statistics in NS. Correlated phantom states can be valid Monte Carlo observations of an expectation while being invalid as race participants. To our knowledge, phantom states have not previously been used in this direct way to improve the evidence calculation itself.

The contributions are as follows.

- First, we formulate NS shrinkage as exponential races of lineages, introducing the *race tree*, which leads to a simplified dynamic nested sampling formulation.
- Second, we present a Bayesian formalism for NS which handles plateaus.
- Third, we introduce phantom-conditioned NS.
- Finally, we present JAXNS v3, an open-source implementation of phantom-boosted NS.

The paper is structured in the following order. Section 2 introduces the race tree formulation. Section 3 introduces a Bayesian model of which handles plateaus rigorously. Section 4 uses the Bayesian model to develop phantom-conditioned NS. Section 5 describes the JAXNS v3 implementation choices. Section 6 presents benchmarks of accuracy and efficiency of the JAXNS implementation.

2 Classic NS as an exponential race

For a threshold λ , define the strict constrained set and its prior volume by,

$$\mathcal{A}_\lambda \triangleq \{x \in \mathcal{A} : L(x) > \lambda\}, \quad (3)$$

$$X_\lambda \triangleq p(\mathcal{A}_\lambda). \quad (4)$$

When $X_\lambda > 0$, define the strict constrained prior by,

$$\pi_\lambda(dx) \triangleq \frac{\mathbf{1}\{L(x) > \lambda\}}{X_\lambda} p(dx). \quad (5)$$

We shall call a *sample* from a constrained prior a pair $(x_i \sim \pi_\lambda, L_i \triangleq L(x_i))$, with *sample index* i the label of the sample.

Let λ_0 be the *sentinel* likelihood associated with $X_{\lambda_0} = 1$. For increasing distinct likelihood levels,

$$\lambda_0 < \lambda_1 < \dots < \lambda_G, \quad (6)$$

write $X_g \triangleq X_{\lambda_g}$. Nested sampling quadrature estimates the evidence as,

$$Z \approx \sum_{g=1}^G \lambda_g (X_{g-1} - X_g). \quad (7)$$

The likelihood levels in Eq. 6 are an arbitrary *schedule* of values, and the goal of NS is to assign each level λ_g a random variable X_g with known distribution. We call the subscript g the *block index*. In addition, an auxiliary output of NS is a set of weighted samples drawn from the posterior, with weights,

$$w_g \triangleq \lambda_g (X_{g-1} - X_g), \quad (8)$$

except in the case of samples that fall in likelihood plateaus. The weights for those will be shown later.

Let us introduce the concept of a race tree. Let there be a set of N samples, as well as a sentential sample with sample index 0 given likelihood value $L_0 = \lambda_0$. These samples form a race tree if each sample (excluding the sentinel sample) is an independent draw from some constrained prior whose contour is also contained in the set of samples,

$$\forall i \in (1 \dots N) \quad (9)$$

$$\exists p(i) \in (0 \dots N) \quad (10)$$

$$x_i \sim \pi_{\lambda_{p(i)}}. \quad (11)$$

In this case we say that $p(i)$ is the parent of i , denoting this as $p(i) \rightarrow i$, and that these samples form a race tree. A *lineage* is a maximal chain of parent links. We shall

refer to these samples as *classic samples*, to differentiate them from phantom samples later. The samples are non-strictly monotonic increasing,

$$\lambda_0 = L_0 < L_1 \leq \dots \leq L_N, \quad (12)$$

and we define a *block* for a level λ_g as,

$$\mathcal{B}_g \triangleq \{i : L_i = \lambda_g\}, \quad m_g \triangleq |\mathcal{B}_g|. \quad (13)$$

A block with $m_g > 1$ represents a plateau, where the ordering of samples in the block is not unique.

The out-degree of sample i is

$$d_i \triangleq \#\{j : p(j) = i\}. \quad (14)$$

The sentinel must have out-degree $d_0 > 0$ in order for the race tree to not be empty.

The crucial observation in NS is that a single sample generated from a parent $p(i) \rightarrow i$ shrinks the prior volume uniformly,

$$r_{p(i) \rightarrow i} \sim \mathcal{U}[0, 1] \quad (15)$$

$$-\log r_{p(i) \rightarrow i} \sim \text{Exp}(1). \quad (16)$$

Define *race time* $s \triangleq -\log X$ and *race rank* $r = \exp -s$, and interpret the race tree as a set of lineages equipped with unit-rate clocks incrementally racing from one sample to the next sample. For any race $i-1 \rightarrow i$ there are K_i active racing lineages. We have that the race time for $i-1 \rightarrow i$ is,

$$s_{i-1 \rightarrow i} \sim \text{Exp}(K_i) \quad (17)$$

$$r_{i-1 \rightarrow i} \triangleq e^{-s_{i-1 \rightarrow i}}. \quad (18)$$

As is well-known, $r_{i-1 \rightarrow i} \sim \text{Beta}(K_i, 1)$.

We start off with $K_1 = d_0$. After the race $i-1 \rightarrow i$ is done, that lineage segment $p(i) \rightarrow i$ terminates and d_i child lineage segments begin at the corresponding strict contour. The active lineage count therefore updates as

$$K_{i+1} = K_i - 1 + d_i. \quad (19)$$

An interesting aspect of this formulation is that there is no notion of live points, nor of a specific order that samples must be generated, beyond the race tree structure requirements. A new child can be generated from any existing strict contour, and the only thing that must be tracked is the out-degree of its parent. This opens up the potential for parallel processing novelties.

Proposition 1 (Race-tree shrinkage). *Suppose that there are K_i active lineages participating in race $i-1 \rightarrow i$, each represented by a unit-rate clock. Then $s_{i-1 \rightarrow i} \sim \text{Exp}(K_i)$, and afterwards the rate update is updated by Eq. 19.*

Proof. The minimum of independent exponential clocks is exponential with rate equal to the sum of the rates. Once the arriving sample i is observed, its lineage segment no longer contributes to the next segment of the race and its d_i child lineage segments begin. This changes the active rate by $-1 + d_i$, giving Eq. 19. $\square$

In classic races there are no ties, however in the presence of a plateau, $m_g > 1$, we have to have ties. The correct way to handle this is to introduce a nuisance winner ordering parameter that decides the order of winners, and that the race is not over until all winners have been accounted for. While the race is proceeding the race clocks are still ticking, hence each sample k of block $\mathcal{B}_g$ gets a different race rank, $r_{g-1 \rightarrow g}^{(k)}$ falling within the atom $L = L_g$. It can easily be shown that the final sample in the plateau has known rank,

$$r_{g-1 \rightarrow g}^{(m_g)} \sim \text{Beta}(K_g - m_g + 1, m_g), \quad (20)$$

however since all plateau samples are necessarily inside the atom, we have that $r_{g-1 \rightarrow g}^{(m_g)} \neq r_{g-1 \rightarrow g}$ almost surely. This is depicted in Figure 1.

It will be helpful to define the following quantities,

$$p_{<g} \triangleq \mathbb{P}(L_g > L > L_{g-1}) \quad (21)$$

$$p_{=g} \triangleq \mathbb{P}(L = L_g) \quad (22)$$

$$p_{>g} \triangleq \mathbb{P}(L > L_g) \quad (23)$$

$$= \frac{X_g}{X_{g-1}} \quad (24)$$

$$= r_{g-1 \rightarrow g}, \quad (25)$$

where Eq. 21 defines the race interval, Eq. 22 defines a possible atom, and Eq. 23 is the end point of the race. As shown in Figure 1 the presence of an atom censors us from learning the end point $p_{>g}$. Thus, when there is a plateau we know that all the plateau samples *finish* the race, just not *when*,

$$p_{>g} \leq r_{g-1 \rightarrow g}^{(m_g)} \leq p_{>g} + p_{=g}. \quad (26)$$

We call this loss of knowledge *plateau censorship*.

3 Bayesian shrinkage model

We propose a Bayesian model that will simultaneously resolve the problem of plateau censorship, and allow us to incorporate phantom samples into shrinkage. Define the block probability vector,

$$p_g = (p_{>g}, p_{=g}, p_{<g}), \quad (27)$$

where clearly $|p_g| = 1$. Consider a block with K_g incoming active lineages and m_g block samples. Then, these define observations,

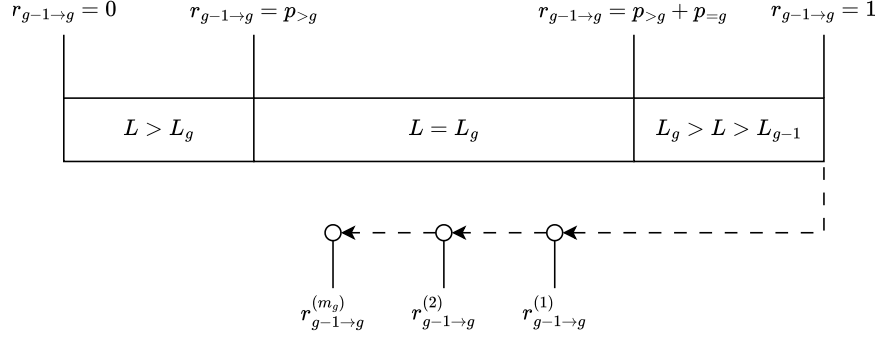


Fig. 1 The presence of an atom $L = L_g$ causes us to lose exact knowledge of $p_{>g}$, since $r_{g-1 \rightarrow g}^{(m_g)} \in [p_{>g}, p_{>g} + p_{=g}]$. The atom can be seen as an interval in race rank just before finishing the race where samples loose identity.

- $K_g - m_g$ samples lie past the atom,
- m_g samples fall in the atom,
- 0 samples fall in the race interval.

These are independent categorical observations, therefore the likelihood of p_g is multinomial,

$$\mathcal{L}(p_g) \propto p_{>g}^{K_g - m_g} p_{=g}^{m_g}. \quad (28)$$

We set a Dirichlet prior,

$$p(p_g) = \text{Dirichlet}(1, \epsilon_g, 1 - \epsilon_g), \quad (29)$$

which can be done by identifying the marginal of the Dirichlet, $\text{Beta}(a_i, \sum_{j \neq i} a_j)$, with uniform shrinkage prior, $\mathcal{U}[0, 1] \equiv \text{Beta}(1, 1)$. The variable ϵ_g is a free parameter and should represent our belief of $\mathbb{E}[p_{=g}/(1 - p_{>g})]$. A fairly neutral choice would be, if $m_g = 1$ then set $\epsilon_g = 10^{-6}$, or else set $\epsilon_g = 1/2$ if $m_g > 1$. Using conjugacy of the Dirichlet and multinomial distributions the posterior becomes exactly,

$$p_g \mid K_g, m_g \sim \text{Dirichlet}(a_{>g}, a_{=g}, a_{<g}), \quad (30)$$

$$a_{>g} = K_g - m_g + 1, \quad (31)$$

$$a_{=g} = m_g + \epsilon_g, \quad (32)$$

$$a_{<g} = 1 - \epsilon_g \quad (33)$$

therefore the posterior marginals for $p_{>g}$ and $p_{=g}$ are,

$$p_{>g} \mid K_g, m_g \sim \text{Beta}(a_{>g}, a_{=g} + a_{<g}) \quad (34)$$

$$p_{=g} \mid K_g, m_g \sim \text{Beta}(a_{=g}, a_{>g} + a_{<g}) \quad (35)$$

Comparing Eq. 34, $\text{Beta}(K_g - m_g + 1, m_g + 1)$ to Eq. 20, $\text{Beta}(K_g - m_g + 1, m_g)$, we see the difference, which proves censorship in a rigorous way.

Evidence calculation can be done using,

$$X_g = X_{g-1} p_{>g}, \quad (36)$$

which admits producing samples of the evidence. Note, when we wish to assign posterior mass to plateau samples in a block this is done by equally assigning atom mass to each sample,

$$w_g^{(k)} = \frac{\lambda_g}{m_g} X_{g-1} p_{=g}, \quad k \in \mathcal{B}_g, \quad (37)$$

whereas the posterior mass assigned to a non-plateau sample, mirroring Eq. 8, is,

$$w_g = \lambda_g X_{g-1} (1 - p_{>g}). \quad (38)$$

4 Phantom-conditioned shrinkage

We are now equipped to use information from phantom samples to improve shrinkage. Let $\mathcal{P}_i$ be phantom samples associated with sample i , which we'll call a *cluster* of phantom samples. They are correlated draws from $\pi_{\lambda_{p(i)}}$ produced in the process of realising sample i . Typically, this is because a Markov chain Monte Carlo (MCMC) approach is used to produce iid samples from $\pi_{\lambda_{p(i)}}$. Since phantom samples are not independent they cannot participate in the race tree, however we are perfectly well able to use them to construct consistent unbiased MC estimates of expectations under $\pi_{\lambda_{p(i)}}$. We thus seek a procedure for sampling from $p_g \mid K_g, m_g, \bigcup_i \mathcal{P}_i$.

An important assumption we make is that phantom samples *between* clusters are approximately independent, which is justified since initial chain points are randomly selected, while within-cluster phantom samples are allowed to be arbitrarily correlated. In fact, it's possible to redefine phantom clusters as any disjoint decomposition of phantom samples where each cluster is drawn from within the same parent, and samples from different clusters are independent. As will be shown below, it is best to make clusters as small as possible such that this is true. Hence, if a single cluster can be split into two disjoint clusters where independence between the two clusters is approximately true, then it is favourable from the perspective of information utilisation to do this.

Lets define, per cluster c , block interval counts,

$$A_{cg} = \sum_{(x,L) \in \mathcal{P}_c} \mathbf{1}\{L(x) > L_{g-1}\} \mathbf{1}\{L_{p(c)} \leq L_{g-1}\} \quad (39)$$

$$B_{cg} = \sum_{(x,L) \in \mathcal{P}_c} \mathbf{1}\{L(x) > L_g\} \mathbf{1}\{L_{p(c)} \leq L_{g-1}\} \quad (40)$$

$$E_{cg} = \sum_{(x,L) \in \mathcal{P}_c} \mathbf{1}\{L(x) = L_g\} \mathbf{1}\{L_{p(c)} \leq L_{g-1}\}. \quad (41)$$

Then an MC estimate of p_g is,

$$\hat{p}_g = \frac{\sum_c (B_{cg}, E_{cg}, A_{cg} - B_{cg} - E_{cg})^\top}{\sum_c A_{cg}}. \quad (42)$$

Now, if the phantom samples were independent then the observable $\sum_c A_{cg} \hat{p}_g$ would satisfy the multinomial likelihood. In this case, the exact same machinery as in Section 3 could be used, and phantom-conditioned posterior shrinkage would be a Dirichlet distribution. However, as they are not independent the posterior has no closed form.

We attempted initially to try to use a multinomial likelihood with an effective count model, however effective count is hard to infer and wrongly takes into account correlations in joint shrinkage. This led to underestimated log-evidence variance in the best of cases.

We instead propose an MC based approach that approaches the Dirichlet posterior in the limit of independent Phantom samples. To do this we will use the gamma construction of Dirichlet random variables to rewrite the race-induced posterior, Eq. 30,

$$M_{>g} \sim \text{Gamma}(a_{>g}, 1) \quad (43)$$

$$M_{=g} \sim \text{Gamma}(a_{=g}, 1) \quad (44)$$

$$M_{<g} \sim \text{Gamma}(a_{<g}, 1) \quad (45)$$

$$\frac{(M_{>g}, M_{=g}, M_{<g})^\top}{M_{>g} + M_{=g} + M_{<g}} \sim \text{Dirichlet}(a_{>g}, a_{=g}, a_{<g}). \quad (46)$$

Now consider how we would condition on iid phantom samples. In this case, we would decompose each phantom cluster into singletons, since this would be the most irreducible decomposition that maintains inter-cluster independence. Then, we would have that $A_{cg}, B_{cg}, E_{cg} \in \{0, 1\}$, that is each cluster would contribute at most one count observation to each block. Define a cluster weight v_c and then define,

$$M'_{>g} = M_{>g} + \sum_c v_c B_{cg} \quad (47)$$

$$M'_{=g} = M_{=g} + \sum_c v_c E_{cg} \quad (48)$$

$$M'_{<g} = M_{<g} + \sum_c v_c A_{cg} \quad (49)$$

$$p_g = \frac{(M'_{>g}, M'_{=g}, M'_{<g})^\top}{M'_{>g} + M'_{=g} + M'_{<g}}. \quad (50)$$

Using additivity of identical-scale gamma random variables, we can then identify the natural cluster weight as,

$$v_c \sim \text{Gamma}(1, 1). \quad (51)$$

With this choice, singleton clusters would exactly produce the Dirichlet posterior.

Now, consider what happens when cluster sizes are not singleton, that is we have irreducible correlation structure. In this case, the gamma's do not add as the scales are not identical. Under multiplication gamma variance scales quadratically, compared to linearly under addition, whereas the mean scales linearly under both multiplication and addition. Thus, the net effect of correlated clusters is to inflate variance, while preserving the mean, which is precisely what we expect correlation to do.

An important well-known consideration for conditioning on MC counts is that there should be enough independent counts to properly capture the variance of outcomes. This why using sparsely populated histograms to draw conclusions is a bad idea. We impose a minimum participating cluster count criterion in order for a block to condition on phantom count information. Specifically, we use a Kish-like quantity to measure effective participating cluster count,

$$C_g^{\text{Kish}} = \frac{(\sum_c A_{cg})^2}{\sum_c A_{cg}^2}, \quad (52)$$

and then define a minimum threshold for condition, say $C_{\min} = 20$, and hence,

$$M'_{>g} = M_{>g} + \mathbf{1}\{C_g^{\text{Kish}} \geq C_0\} \sum_c v_c B_{cg} \quad (53)$$

$$M'_{=g} = M_{=g} + \mathbf{1}\{C_g^{\text{Kish}} \geq C_0\} \sum_c v_c E_{cg} \quad (54)$$

$$M'_{<g} = M_{<g} + \mathbf{1}\{C_g^{\text{Kish}} \geq C_0\} \sum_c v_c A_{cg} \quad (55)$$

$$p_g = \frac{(M'_{>g}, M'_{=g}, M'_{<g})^\top}{M'_{>g} + M'_{=g} + M'_{<g}}. \quad (56)$$

This provides a consistent generative shrinkage model that properly takes into account the race tree and phantom samples. The shrinkage at block g is the the component $p_{>g}$, while the plateau atom size is $p_{=g}$. It is simple to calculate as the per-cluster counts are computed once, and then each draw from the joint posterior is just a set of draws from gamma distributions followed by reduction.

Phantom-conditioning requires retained phantom states to be stationary enough that their marginal expectation under the parent contour is correct. The above formulation cannot correct for non-stationary phantom samples, thus a burn-in is suggested before collecting phantoms.

5 JAXNS v3 implementation

The race tree permits new children to be generated from within any existing sample contour. This decouples correct lineage counting from the strategy of sample generation, and allows us to explore optimal allocation targets as an experimental design question. Algorithm 1 shows the core algorithm of JAXNS. There is an outer loop which sets a lineage allocation target as a function of likelihood, $K_*^k : \mathbb{R}_+ \rightarrow \mathbb{N}$. There

is an inner loop which, given K_*^k , generates samples until a depth condition is met. Sample generation can be embarrassingly parallelised. The outer loop proceeds until a goal condition is met. During the inner loop samples are identified whose active lineage counts do not meet the allocation target, and a sampling contour is chosen with high probability of increasing the active lineage count. A strong way to choose a parent block with a high probability to reach block g' is choose block g with probability $(X_{g'}/X_g) \mathbf{1}\{X_{g'} < X_g\}$. The depth condition captures when there is little utility in continuing with the current allocation target, K_*^k . The depth conditions available in JAXNS are,

$$\text{small remaining evidence,} \quad \frac{L_G X_G}{\sum_g^G L_g (X_{g-1} - X_g) + L_G X_G} < \tau_Z \quad (57)$$

$$\text{small remaining posterior mass,} \quad \frac{L_g X_g}{\max_{g'} (L_{g'} X_{g'})} < \tau_{\text{post}} \quad (58)$$

The goal condition is user specified, and can be any condition computable from the samples, for example until a target effective sample size or evidence uncertainty is met.

The allocation targets define how likelihood evaluation work is dedicated per outer loop. The available allocation targets in JAXNS are,

$$\text{Depth-first uniform,} \quad K_*^k(L) = d_0 + k\Delta K, \quad (59)$$

$$\text{Evidence improving,} \quad K_*^k(L) = d_0 + k\Delta K \bar{U}^Z(L), \quad (60)$$

$$\text{Posterior improving,} \quad K_*^k(L) = d_0 + k\Delta K \bar{U}^P(L), \quad (61)$$

where $\Delta K > 0$, and $\bar{U}^Z$ and $\bar{U}^P$ are defined below.

Depth-first uniform allocation explores higher likelihood structure before shallower contours are over-refined. Each outer loop costs similar amounts of work, and progressively better resolves modes. This results in better cost management, rather than choosing an initial arbitrarily large root lineage count, d_0 .

Evidence improving allocation seeks to allocate new samples to contours such that there is a high probability of reducing the evidence variance, by maximising expected improvement. A key observation is that a new child generated from parent block g changes the parent out-degree as $d_g \rightarrow d_g + 1$, Eq. 19. The probability of that sample reaching L_h is given by,

$$T_{gh}^Z \triangleq \mathbb{P}_{x \sim \pi_{L_g}}(L(x) > L_h) = \frac{X_h}{X_g} \mathbf{1}\{h > g\}. \quad (62)$$

That is, the probability that we add one additional active lineage count to blocks $g+1$ to l is T_{gh}^Z .

We will define utility function as the expected reduction in variance. To simplify calculation, we will use a first-order delta approximation to the log-evidence variance in terms of the log-shrinkage variance. Structurally, we have $\text{Var}[f; \theta] \approx \sum_{\theta} (\partial_{\theta} f)^2 \text{Var}[\theta]$. Applying this to $\log Z$ given by Eq. 7 with block shrinkage $p_{>h} \sim \text{Beta}(\alpha_h, \beta_h)$, and

recalling that $X_h = p_{>h} X_{h-1} = \prod_j^h p_{>j}$ we have,

$$B_h \triangleq \frac{\partial \log Z}{\partial \log p_{>h}} = \frac{L_h X_h - \sum_{j>h} L_j (X_{j-1} - X_j)}{Z}, \quad (63)$$

$$\text{Var}[\log p_{>h}] = \psi_1(\alpha_h) - \psi_1(\alpha_h + \beta_h), \quad (64)$$

where ψ_1 is the tri-gamma function. The first term in Eq. 63 is the contribution from changing the block h , and the second term is the accumulated downstream effect because $p_{>h}$ appears in every later term. Now the effect of adding one extra active lineage is that $\alpha_h \rightarrow \alpha_h + 1$, so the reduction in variance of log-evidence under this delta approximation is given by,

$$\Delta \text{Var}[\log Z; h] \approx B_h^2 \left(\frac{1}{\alpha_h^2} - \frac{1}{(\alpha_h + \beta_h)^2} \right), \quad (65)$$

where we used the relationship $\psi_1(z) - \psi_1(z+1) = 1/z^2$. Now, the utility is the expectation of Eq. 65 over the transition probability, Eq. 62,

$$U_g^Z = \frac{1}{X_g} \sum_{h>g} X_h B_h^2 \left(\frac{1}{\alpha_h^2} - \frac{1}{(\alpha_h + \beta_h)^2} \right). \quad (66)$$

This can be computed efficiently in two cumulative reductions.

Posterior improving allocation seeks to increase the effective sample size, by allocating samples low-information shells defined by the half-open likelihood intervals, $(L_{h-1}, L_h]$. A key observation is that a uniform draw within contour L_g will land in shell h with probability,

$$T_{gh}^P \triangleq \mathbb{P}_{x \sim \pi_{L_g}}(L_{h-1} < L(x) \leq L_h) \approx \frac{X_{h-1} - X_h}{X_g} \mathbf{1}\{h > g\}. \quad (67)$$

Recalling the posterior shell masses, Eq. 8, let us use Kish's effective sample size as a proxy for effective sample size,

$$\text{ESS}^{\text{Kish}} = \frac{W^2}{Q} \quad (68)$$

$$W \triangleq \sum_{j=1}^G w_j, \quad (69)$$

$$Q \triangleq \sum_{j=1}^G w_j^2. \quad (70)$$

If a new sample lands in shell h , it splits the shell into two sub-shells. In this case W is unchanged under additivity, and only Q changes. Writing $u \sim \mathcal{U}[0, 1]$ for the

fraction of the shell mass assigned to the lower sub-shell, the old contribution w_h^2 to Q is replaced by $u^2 w_h^2 + (1-u)^2 w_h^2$. Thus,

$$\text{ESS}_h^{\text{Kish}}(u) = \frac{W^2}{Q - 2u(1-u)w_h^2}. \quad (71)$$

If we marginalise over u the change of ESS^{Kish} due to splitting in h we get,

$$\Delta \text{ESS}_h^{\text{Kish}} \triangleq \int_0^1 \left(\text{ESS}_h^{\text{Kish}}(u) - \text{ESS}^{\text{Kish}} \right) du \quad (72)$$

$$= W^2 \left[\frac{\tan^{-1} \left(\sqrt{w_h^2 / (2Q - w_h^2)} \right)}{\sqrt{(Q - w_h^2/2)(w_h^2/2)}} - \frac{1}{Q} \right], \quad (73)$$

with the continuous limiting value $\Delta \text{ESS}_h^{\text{Kish}} \rightarrow 0$ as $w_h \rightarrow 0$. A simple conservative approximation is obtained by replacing the denominator with its expectation,

$$\Delta \text{ESS}_h^{\text{Kish}} \approx \frac{W^2}{Q - w_h^2/3} - \frac{W^2}{Q}. \quad (74)$$

This is conservative because $1/x$ is convex so $\mathbb{E}[1/x] \geq 1/\mathbb{E}[x]$. Now the utility is found by marginalising over the shell transition probabilities that a child from parent block g lands in that shell h gives the posterior-improving utility,

$$U_g^P = \frac{1}{X_g} \sum_{h>g} (X_{h-1} - X_h) \Delta \text{ESS}_h^{\text{Kish}}. \quad (75)$$

In order to convert these block utilities into allocation targets we normalise utility to unit peak and represent in terms of likelihood via linear interpolation, $\bar{U}(L)$, then allocate $k\Delta K$ proportionally,

$$K_*^k(L) = d_0 + k\Delta K \bar{U}(L), \quad (76)$$

The statistical theory requires each classic child to be approximately sampled from the strict constrained prior of its parent. The JAXNS constrained sampler uses one-dimensional slice sampling (Neal, 2000) from a seed point x^0 , as shown in Algorithm 2. Each Markov transition explores the constrained prior, with a total of T steps per chain. A single step involves choosing a direction from a symmetric distribution $\hat{n} \sim \mathcal{D}$ independent of the current chain. That is, all directions in the chain should come from the same distribution. Given a direction $\hat{n}$ and chain point x^i a trajectory that brackets x^i along the direction is chosen. The trajectory must have end points that land outside the contour, and the end points should be independent from x^i . Then, constrained sampling is performed along this trajectory, which can be done in any manner. A common approach is to randomly pick a point along the trajectory and greedily shrink the trajectory if the point falls outside the slice.

Algorithm 1 The JAXNS core algorithm with dynamic lineage allocation, outer goal terminating loop, and inner depth termination loop.

Require: root lineage count d_0 , sentinel likelihood λ_0

```

1: Sample  $(x_i, L_i, \mathcal{P}_i) \sim \pi_{\lambda_0}$  for  $i \in \{1 \dots d_0\}$ 
2: Set  $N = d_0$ 
3: Set outer iteration  $k = 0$ 
4: while  $\neg$  goal do
5:   Choose allocation target function  $K_*^k$ .
6:   while  $\neg$  depth do
7:     Count lineages  $K_i$  for  $i \in \{1 \dots N\}$ .
8:     Choose parents  $\mathcal{Q} \subseteq \{j : L_j < L_i, K_i^k < K_*(L_i)\}$ .
9:     for  $i \in \mathcal{Q}$  do
10:      Set  $N \rightarrow N + 1$ 
11:      Set  $j \rightarrow N$ 
12:      Choose seed  $L(x_j^0) > L_i \in \{1 \dots N\}$ .
13:      Sample  $(x_j, L_j, \mathcal{P}_j) \sim \pi_{L_i}$  from  $x_j^0$ .
14:      Set  $d_i \rightarrow d_i + 1$ 
15:     end for
16:     Compute depth.
17:   end while
18:   Set  $k \rightarrow k + 1$ .
19:   Compute goal.
20: end while
21: return  $\{(x_i, L_i, \mathcal{P}_i, d_i) : i \in \{1 \dots N\}\}$ 

```

Note 1: For a parent, if there are no possible seeds from it, i.e. due to a plateau, then we change the parent to 0 (the sentential).

Note 2: We do not need to keep track of parents for each sample, since the algorithm only needs to know out-degree. If parent blocks were desired, they could be uniquely reconstructed from sorted likelihoods and out-degrees.

In choosing $\mathcal{D}$, JAXNS makes available two choices: an isotropic Gaussian, and an ellipsoidal Gaussian. For the later, a Gaussian mixture model of the posterior is constructed based on samples collected so far. The distribution $\mathcal{D}$ first samples a Gaussian component proportional to its integrated volume, and then samples the direction from that Gaussian.

Concerning the construction of the trajectory for a single step in the chain, and sampling from it, there are two available methods in JAXNS. The first is perfect bracketing along a straight line, where the endpoints are chosen to be the intersection with the bounds of the unit-hypercube representing the prior homogeneous measure. This is the maximal interval possible and can be determined without any likelihood evaluations. Constrained sampling along this trajectory proceeds with uniform sampling and greedy trajectory shrinkage which exponentially contracts towards the current chain point, where the slice is necessarily satisfied. This approach trades off the zero-cost trajectory determination with extra likelihood evaluations to uniformly sample

Algorithm 2 The JAXNS constrained sampler using one-dimensional slice sampling with general trajectory building and uniform trajectory sampling.

Require: direction kernel $\mathcal{D}$, contour λ , seed point x^0 , steps per acceptance T , burn-in B .

```

1: Set  $i = 0$ .
2: while  $i < T$  do
3:   Choose direction,  $\hat{n} \sim \mathcal{D}$ .
4:   Construct a trajectory  $I$  that passes through  $x^i$  along  $\hat{n}$ , with end points outside
     the slice and independent from  $x^i$ .
5:   Uniformly sample  $x^{i+1} \sim \pi_\lambda$  along  $I$ .
6:   Set  $i \rightarrow i + 1$ .
7:   Set done =  $L(x^i) > \lambda$ .
8: end while
9: Set phantom cluster  $\mathcal{P} = \{(x^k, L(x^k)) : k \in \{B \dots T - 1\}\}$ .
10: return  $(x^T, L(x^T), \mathcal{P})$ 

```

the trajectory. However, in our experience this extra cost is negligible, and is compensated by better prior exploring. One problem with this approach is that it can lead to increased chance of mode evaporation, since a chain can easily move between modes. The probability of tunnelling between modes depends on the geometry of the problem, and can be ameliorated by the adaptive lineage allocation. Depth-first uniform growth in particular can rediscover modes.

Another method of placing and sampling the trajectory is to use Galilean sampling, where Hausdorff reflections off the hard likelihood constraints are used to preserve reversibility, and thus detailed balance, and keep the trajectory inside the slice. The trajectory is constructed from the seed point x^i in both directions $\hat{n}$ and $-\hat{n}$ until the momentum vector is anti-aligned with the initial direction, a so-called *U-turn*. Algorithm 3 describes creating a single leg of the trajectory from an initial point in one direction. The full trajectory is generally non-straight and bounces off the hard boundary constraints. Since the trajectory *should* be contained within the slice, uniform sampling without trajectory shrinkage is used. This is done by selecting a segment of the trajectory proportionally to segment length, and then uniform sampling a point in the segment.

JAXNS is designed for easy heterogeneous parallel and distributed likelihood evaluation scaling from a laptop to clusters with thousands of nodes. A central process performs the coordination of the core algorithm, while a set of workers perform likelihood evaluations. Workers receive serialized likelihood models and are scheduled by load balancing with overlapping computation. Communication is achieved with ZMQ, while serialisation is achieved with the Python’s ‘pickle’ functionality. Arguments to likelihood model should consist of arbitrarily nested trees of literals and arrays. This allows constrained-sampling calls from different parent contours to overlap in wall-clock time. The race-tree formulation is key to this asynchronous setting because a completed child only updates the out-degree of its parent which is known before dispatching a batch of sample generation.

Algorithm 3 Galilean single-leg trajectory building by bracketing the contour endpoint and Hausdorff reflecting from the inner point until a U-turn.

Require: Contour λ , initial point x^0 with $L(x^0) > \lambda$, initial direction $\hat{n}^0$ with $|\hat{n}^0| = 1$, initial step-size Δt .

```

1: Set  $i = 0$ .
2: while  $\hat{n}^i \cdot \hat{n}^0 \geq 0$  do
3:   Propose step forward  $x' \rightarrow x^i + \Delta t \hat{n}^i$ .
4:   if  $L(x') \leq \lambda$  then Backtrack into contour
5:     Set  $\Delta t \rightarrow \Delta t/2$ .
6:     Propose step back  $x' \rightarrow x^i + \Delta t \hat{n}^i$ 
7:     while  $L(x') \leq \lambda$  do
8:       Adapt step-size  $\Delta t \rightarrow \Delta t/2$ .
9:       Propose step back,  $x' \rightarrow x^i + \Delta t \hat{n}^i$ .
10:    end while
11:    Reflect  $\hat{n}^{i+1} \rightarrow \hat{n}^i - 2(\hat{n}^i \cdot \nabla L(x')) \nabla L(x')$ .
12:    Normalise  $\hat{n}^{i+1} \rightarrow \hat{n}^{i+1}/|\hat{n}^{i+1}|$ .
13:    Set  $x^{i+1} \rightarrow x'$ .
14:    Set  $i \rightarrow i + 1$ .
15:  else Step out of contour
16:    Set  $\Delta t \rightarrow 2\Delta t$ .
17:    Set temporary  $x'' \rightarrow x'$ 
18:    Propose step forward  $x' \rightarrow x^i + \Delta t \hat{n}^i$ 
19:    while  $L(x') > \lambda$  do
20:      Adapt step-size  $\Delta t \rightarrow 2\Delta t$ .
21:      Set temporary  $x'' \rightarrow x'$ 
22:      Propose step forward,  $x' \rightarrow x^i + \Delta t \hat{n}^i$ .
23:    end while
24:    Reflect  $\hat{n}^{i+1} \rightarrow \hat{n}^i - 2(\hat{n}^i \cdot \nabla L(x'')) \nabla L(x'')$ .
25:    Normalise  $\hat{n}^{i+1} \rightarrow \hat{n}^{i+1}/|\hat{n}^{i+1}|$ .
26:    Set  $x^{i+1} \rightarrow x''$ .
27:    Set  $i \rightarrow i + 1$ .
28:  end if
29: end while
30: return ( $\{x^j : j \in \{0 \dots i\}\}, \Delta t$ )

```

Note 1: we cannot keep phantom samples from this trajectory building because they do not uniformly sample from π_λ .

6 Experimental protocol

The empirical results in this version are placeholders for new JAXNS benchmarks. All reported experiments will average over many independent random seeds. They will compare baseline race-tree nested sampling, phantom-conditioned shrinkage, dynamic contour allocation, gradient-informed constrained sampling.

Table 1 Placeholder evidence-calibration table.

Problem	Method	Seeds	Mean $z_{\log Z}$	SD $z_{\log Z}$	RMSE $\log Z$	Likelihood
To be filled	Baseline	To be filled	To be filled	To be filled	To be filled	To be filled
To be filled	Phantom-conditioned	To be filled	To be filled	To be filled	To be filled	To be filled
To be filled	Dynamic allocation	To be filled	To be filled	To be filled	To be filled	To be filled
To be filled	Gradients and phantoms	To be filled	To be filled	To be filled	To be filled	To be filled

6.1 Correctness and calibration

The first experiment class validates the conditioning mechanism on problems for which the evidence or survival curve is known. The key evidence diagnostic is

$$z_{\log Z} \triangleq \frac{\widehat{\log Z} - \log Z_{\text{ref}}}{\widehat{\sigma}_{\log Z}}. \quad (77)$$

where $\widehat{\log Z}$ and $\widehat{\sigma}_{\log Z}$ come from MC-based shrinkage. We'll also compare the expectation based estimates to these MC-based estimates. Each seed will produce a $z_{\log Z}$, and we should expect mean $z_{\log Z}$ near zero and standard deviation near one over repeated runs. The same experiments will report bias in $\widehat{\log Z}$, empirical root-mean-square error, and the relationship between reported and empirical uncertainty. The per block ρ_g curves will be shown along side the fit.

Plateau tests will include likelihoods with equality atoms. These tests will measure recovery of equality masses in addition to the above diagnostics. The expected qualitative result is that phantom equality counts improve atom-mass inference without biasing shrinkage.

6.2 Evidence-efficiency benchmarks

The second experiment class uses difficult evidence benchmarks chosen to stress constrained-prior exploration. For each problem and method setting, we will report likelihood evaluations per effective sample size, and wall-clock timing where meaningful. We will summarize cost-error tradeoffs with Pareto plots of RMSE against likelihood evaluations. Since log-evidence variance is inversely proportional to effective sample size a method is considered dominated when MSE times number of likelihood evaluations is lower for another method.

6.3 Posterior-estimation benchmarks

The third experiment class measures posterior quality on problems with tricky modes and high-resolution reference posterior samples. For a posterior sample set $\widehat{\Pi}$ and reference sample set Π_{ref} , the main discrepancy will be an MC estimate of Wasserstein distance,

$$\widehat{W}(\widehat{\Pi}, \Pi_{\text{ref}}). \quad (78)$$

Placeholder for Pareto-front plots of evidence error versus likelihood evaluations.

Fig. 2 Planned Pareto-front summary for evidence benchmarks.

Placeholder for posterior Wasserstein distance versus likelihood evaluations per effective sample.

Fig. 3 Planned posterior-quality summary for multimodal benchmarks.

These metrics are kept separate from evidence calibration because a method can improve evidence uncertainty without improving posterior representation.

7 Conclusions

This paper reformulates NS shrinkage as racing lineages, which decouples the shrinkage arithmetic from the sample generation process. We then introduce a rigorous Bayesian model for dealing with plateaus and conditioning on phantom samples. The endpoint of a plateau is separated from the strict endpoint needed to continue compression, and the Bayesian model is able to account for this. Phantom samples supply correlated Monte Carlo observations of constrained-prior probabilities. The resulting method uses information that Markov-chain constrained samplers already generate, while preserving the order-statistic law of the classic sequence. This framework is implemented in JAXNS v3. Experiments compare the efficiency improvement of the above methods against classic NS.

Supplementary information. Code and data required to reproduce the experiments will be provided in the JAXNS repository at <https://www.github.com/JoshuaAlbert/jaxns>.

Acknowledgments. JA would like to thank John Skilling and Do Kester for discussions around nested sampling, as well as Will Handley, Andrew Fowlie, and Johannes Buchner for pointing out flaws in earlier attempts to use phantom samples.

References

- Buchner J (2023) Nested Sampling Methods. *Statistics Surveys* 17:169–215. <https://doi.org/10.1214/23-SS144>, [arXiv:2101.09675](https://arxiv.org/abs/2101.09675) [stat.CO]
- Neal RM (2000) Slice Sampling. *arXiv e-prints* physics/0009028. [arXiv:physics/0009028](https://arxiv.org/abs/physics/0009028) [physics.data-an]
- Skilling J (2004) Nested Sampling. In: Fischer R, Preuss R, Toussaint UV (eds) *Bayesian Inference and Maximum Entropy Methods in Science and Engineering: 24th International Workshop on Bayesian Inference and Maximum Entropy Methods in Science and Engineering*, pp 395–405, <https://doi.org/10.1063/1.1835238>